

QUESTION 1

(a) Problem Description

During examination periods, college and university students face significant pressure attempting to build and manage study schedules. Most students take between 5 to 7 modules in a semester, each with different levels of difficulty, broad syllabus and unevenly spaced exam dates.

Who suffers from this problem?

Tertiary students across all disciplines suffer from this problem, particularly those taking technical and heavy workload programs where multiple complex exams are written over short periods.

How often does it happen?

This issue recurs at least twice every academic year during mid-semester tests and final examination sessions, usually lasting several weeks.

Why is it worth solving?

Creating a study timetable manually using paper or standard spreadsheets is time-consuming and rigid. If an unexpected event occurs such as illness, an emergency, or an unexpected delay a static paper timetable becomes outdated immediately. As a result, students often resort to last minute cramming or spend too much time studying familiar, easier subjects while neglecting harder ones. An automated timetable generator solves this by dynamically calculating subject priorities and distributing study slots based on difficulty and time remaining before each exam.

(b) Proposed Solution

The proposed solution is an **Adaptive Revision Timetable Generator**. It is an algorithmic tool that constructs a custom, balanced study schedule for students leading up to their exams.

What would it do?

The system takes student input including the list of enrolled modules, the exact date for each exam, a self assessed difficulty rating for each module on a scale of 1 to 5 and the total available daily study hours. Using this information, the algorithm calculates a priority weight for each subject based on its difficulty and how soon the exam takes place. It then assigns daily study sessions into designated time blocks while applying practical constraints , limiting continuous study on a single module to a maximum of 2 hours per day to prevent exhaustion, inserting rest intervals, and recalculating the schedule if study sessions are missed.

Who would use it?

College students, high school students preparing for national final exams and self taught learners working through multiple certifications simultaneously.

(c) Current Approaches vs. Proposed Solution

Currently students manage their revision using paper planners, static calendar apps such as Google Calendar or informal daily decisions.

Paper planners are completely rigid, skipping one day renders the entire plan inaccurate. Standard digital calendar tools allow setting reminders, but they require manual entry for every session and do not contain domain specific logic to automatically balance subject difficulty against exam proximity.

My approach improves upon existing solutions by incorporating dynamic priority scoring and automated slot allocation. It mathematically evaluates which subject requires attention first and automatically adjusts future time slots whenever a student logs a missed or incomplete study block.

QUESTION 2

(a) Inputs, Outputs, and Assumptions

Category	Item Name	Data Type & Description
Inputs	modulesList	Array of records containing ModuleName (String), ExamDate (Date), and DifficultyRating (Integer from 1 to 5).
Inputs	dailyAvailableHours	Real number representing available study hours per day (e.g., 6.0).
Inputs	startDate	Date representing the first day of revision.

Category	Item Name	Data Type & Description
Outputs	timetableSchedule	Array of structured daily slots detailing Date, TimeSlot, ModuleName, and Duration.
Outputs	Subject Summary	Summary report displaying total study hours allocated per subject.
Assumptions	Fixed Exam Dates	1. Daily available study hours remain consistent across the revision period.
Assumptions	Non-negotiable	2. Exam dates are fixed and non-negotiable.
Assumptions	Cognitive Fatigue	3. Maximum continuous study time for any single module on a single day is capped at 2.0 hours.

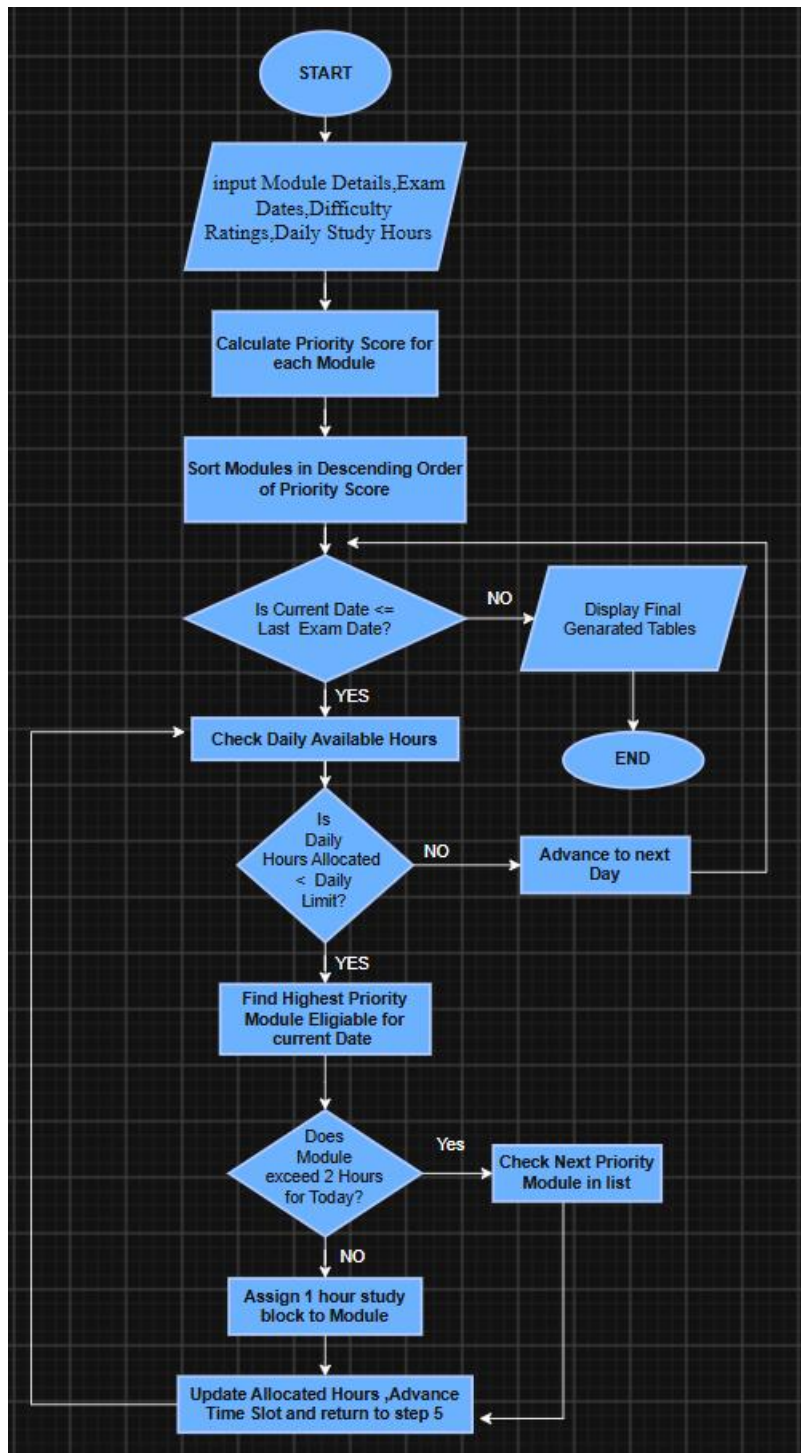
(b) Data Structure Choice & Justification

Chosen Data Structure: An Array of Records used to maintain module priority rankings, alongside an 1D Array of Daily Time Slots for the final generated schedule.

Justification:

To determine which subject should be assigned to an available study block, modules are evaluated using a Priority Score = (Difficulty Rating * 10) / Days Remaining Until Exam. Using an Array of Records sorted by Priority Score (or a Max-Heap Priority Queue) allows the algorithm to immediately inspect and select the most urgent subject in O(1) time. Compared to a Singly Linked List (the alternative considered) a linked list would require linear search traversal O(N) across all modules every time a study slot needs filling. For N modules and M available time blocks a linked list approach results in O(N * M) operations. Sorting an array or maintaining a heap allows instantaneous look ups and efficient updates, making it far better suited for multi-subject timetable generation.

(c) Flowchart / Block Diagram using Draw.io



QUESTION 3

(a) Pseudocode for MAIN Algorithm

ALGORITHM GenerateRevisionTimetable

INPUT:

LOVELLE K MUSADOGERA
DATA STRUCTURES AND ALGORITHMS (677/24/M09)
INNOVATION-BASED ASSIGNMENT
DATA SCIENCE 2.2

modules: Array of Module records (name, examDate, difficulty)

dailyHours: Real

startDate: Date

OUTPUT:

schedule: Array of StudySlot records (date, moduleName, duration)

BEGIN

// Step 1: Initialization

totalModules <- LENGTH(modules)

lastExamDate <- GetLatestExamDate(modules)

totalDays <- DaysBetween(startDate, lastExamDate)

maxSlots <- totalDays * DailySlotsCount(dailyHours)

schedule <- CreateEmptyArray(maxSlots)

// Step 2: Calculate Priority Score for each module

FOR i FROM 0 TO totalModules - 1 DO

daysRemaining <- DaysBetween(startDate, modules[i].examDate)

IF daysRemaining <= 0 THEN

daysRemaining <- 1

END IF

// Priority weight calculation (Difficulty divided by Days Remaining)

modules[i].priorityScore <- (modules[i].difficulty * 10) / daysRemaining

modules[i].allocatedHours <- 0

END FOR

// Step 3: Sort Modules by Priority Score (Descending Order)

CALL SortModulesByPriority(modules, totalModules)

// Step 4: Fill Time Slots Day by Day

currentDate <- startDate

slotIndex <- 0

WHILE currentDate <= lastExamDate DO

dailyAllocatedHours <- 0.0

WHILE dailyAllocatedHours < dailyHours DO

selectedModuleIndex <- -1

// Search for highest priority module that hasn't hit its daily limit

FOR m FROM 0 TO totalModules - 1 DO

IF modules[m].examDate > currentDate THEN

subjectHoursToday <- GetDailySubjectHours(schedule, currentDate,

modules[m].name)

IF subjectHoursToday < 2.0 THEN

selectedModuleIndex <- m

```
        EXIT FOR
    END IF
END IF
END FOR
IF selectedModuleIndex != -1 THEN
    // Allocate a 1-hour study session
    schedule[slotIndex].date <- currentDate
    schedule[slotIndex].moduleName <- modules[selectedModuleIndex].name
    schedule[slotIndex].duration <- 1.0
    modules[selectedModuleIndex].allocatedHours <-
modules[selectedModuleIndex].allocatedHours + 1.0
    dailyAllocatedHours <- dailyAllocatedHours + 1.0
slotIndex <- slotIndex + 1
ELSE
    // If all active modules hit limits, assign a rest/break slot
    schedule[slotIndex].date <- currentDate
    schedule[slotIndex].moduleName <- "REST / REVISION BREAK"
    schedule[slotIndex].duration <- 1.0
    dailyAllocatedHours <- dailyAllocatedHours + 1.0
    slotIndex <- slotIndex + 1
END IF
END WHILE
currentDate <- AddDays(currentDate, 1)
END WHILE
RETURN schedule
END ALGORITHM
```

(b) Pseudocode for Supporting Routine

ALGORITHM SortModulesByPriority

INPUT:

modulesList: Array of Module records

count: Integer

OUTPUT:

modulesList (sorted in-place descending by priorityScore)

BEGIN

// Bubble Sort algorithm customized for descending priority order

FOR i FROM 0 TO count - 2 DO

FOR j FROM 0 TO count - i - 2 DO

LOVELLE K MUSADOGERA

DATA STRUCTURES AND ALGORITHMS (677/24/M09)

INNOVATION-BASED ASSIGNMENT

DATA SCIENCE 2.2

```
    IF modulesList[j].priorityScore < modulesList[j + 1].priorityScore THEN
        // Swap records
        tempModule <- modulesList[j]
        modulesList[j] <- modulesList[j + 1]
        modulesList[j + 1] <- tempModule
    END IF
END FOR
END FOR
RETURN modulesList
END ALGORITHM
```